

Backend API Specification

All endpoints are served by the Express backend on port 4000, accessible via Nginx proxy at the site root.

Base URL: `http://your-server` (Nginx proxies `/api/*` to `backend:4000`)

Content-Type: All requests and responses use `application/json` .

Rate limits (applied per IP by `express-rate-limit`):

- Global: 500 req/min (all endpoints)
- Session creation: 30 req/min (`/api/create-session` only)

Authentication

Game endpoints use **JWT Bearer tokens** (HS512 algorithm). The token is obtained by calling `POST /api` and must be sent in the `Authorization: Bearer <token>` header on all protected endpoints.

JWT claims contain `{ sessionId, playerId }` — the middleware injects these into `req.auth` .

Portal endpoints have **no authentication**. They are designed for internal staff use only; restrict access at the firewall level if needed.

Game Endpoints

POST /api — JWT Handshake (Validate)

Validates a session ID and issues a JWT for the player.

Auth: None

Rate limit: Global 500/min

Request body:

```
{ "sessionId": 202604150 }
```

Field	Type	Required	Notes
sessionId	number or string	Yes	9-digit session code

Success (200):

```
{ "token": "<jwt-string>" }
```

Error responses:

Status	Body	When
400	{ "error": "Invalid session ID" }	sessionId missing or non-numeric
400	{ "error": "Session not found or not valid for today" }	Session doesn't exist or sessionId ≠ today (Eastern time)

500	{ "error": "Internal server error" }	DB error
-----	--------------------------------------	----------

Notes:

- The session must exist in the `Session` table AND its `sessionDate` must match today in the `America/New_York` timezone.
- A new `playerId` is generated (cryptographically random, 1–2B range) if the player doesn't already have one stored client-side.
- Token is signed with `JWT_SECRET` env var using HS512.

POST /api/create-session

Creates a new session record in the database.

Auth: None

Rate limit: 30/min (session creation limiter)

Request body:

```
{ "sessionId": 202512250 }
```

Field	Type	Required	Notes
sessionId	number	Yes	9-digit session ID to create

Success (200):

```
{ "sessionId": 202512250 }
```

Error responses:

Status	Body	When
400	{ "error": "..." }	Missing or invalid sessionId
409	{ "error": "Session already exists" }	Duplicate session ID
500	{ "error": "Internal server error" }	DB error

Notes:

- Used by IT staff to create sessions outside the pre-seeded date range.
- Pre-seeded sessions cover January 2024 – December 2024 (format: `YYYYMMDD0`).

POST /api/start-game

Initializes or resumes a player's session and returns their active route.

Auth: Required (JWT Bearer)

Request body: (empty — player identity from JWT)

```
{}
```

Success (200):

```
{
  "sessionId": "202604150",
  "player": {
    "playerId": 1234567890
  },
  "availableRoutes": [],
  "activeRoute": {
    "routeId": "3",
    "routeNodes": [
      { "id": "0", "text": "Starting point description..." },
      { "id": "1.0", "text": "Stop A description..." }
    ],
    "factNodes": [
      { "id": "4.2", "text": "Did you know...", "pollinator": "Hummingbird" }
    ]
  }
}
```

Field	Notes
sessionId	String version of the session ID from JWT
player.playerId	Numeric player ID from JWT
availableRoutes	Always empty array (legacy field, not populated)
activeRoute	The current route in legacy Route shape, or <code>null</code> if no route assigned yet

Notes:

- Creates a `PlayerSession` row if the player hasn't joined this session before (idempotent).
- If the player has no active route, one is allocated immediately via `getNextRouteForPlayerSession`.
- Routes are assigned randomly, cycling through all 11 routes before repeating.

POST /api/complete-route

Marks the player's current route as complete and returns the next route.

Auth: Required (JWT Bearer)

Request body: (empty — player identity from JWT)

```
{}
```

Success (200):

```
{
  "success": true,
  "message": "Route completed",
  "newActiveRoute": {
    "routeId": "7",
    "routeNodes": [...],
    "factNodes": [...]
  },
  "routesCompleted": 2,
  "completedPollinators": ["Hummingbird", "Bat"]
}
```

Field	Type	Notes
success	boolean	Always true on 200
message	string	Human-readable status
newActiveRoute	Route null	Next route, or null if all routes completed
routesCompleted	number	Total routes completed by this player in this session
completedPollinators	string[]	Names of pollinators discovered so far

Error responses:

Status	Body	When
400	{ "error": "No active route to complete" }	Player has no active RouteCycle
401	{ "error": "Unauthorized" }	Missing/invalid JWT
500	{ "error": "Internal server error" }	DB error

POST /api/add-children-adults

Saves the player's party size survey response.

Auth: Required (JWT Bearer)

Request body:

```
{
  "numChildren": 2,
  "numAdults": 1
}
```

Field	Type	Required	Notes
numChildren	number	Yes	Count of children in group (0–20 typical)
numAdults	number	Yes	Count of adults in group (0–20 typical)

Success (200):

```
{ "success": true }
```

Notes:

- Updates the `PlayerSession` row for this player.
- Only called once per visit — the frontend suppresses the survey after first completion.
- No bounds validation server-side — accepts any numeric value.

POST /api/get-pollinator-names

Returns all pollinators from the database with their spritesheet coordinates.

Auth: Required (JWT Bearer)

Request body: *(empty)*

```
{}
```

Success (200):

```
{
  "success": true,
  "message": "data retrieved successfully",
  "pollinator": [
    { "name": "Ant", "coord": "3.4" },
    { "name": "Bat", "coord": "4.1" },
    { "name": "Bee", "coord": "6.0" }
  ]
}
```

Field	Type	Notes
pollinator	{ name: string, coord: string }[]	Each entry is one route's final node
pollinator[].name	string	Pollinator name (same as the route name)
pollinator[].coord	string	Spritesheet key of the route's last node (e.g. "3.4")

Notes:

- Used by both `home/page.tsx` (to count total pollinators) and `pollinator-collection/page.tsx` (to display silhouettes for undiscovered pollinators).
- Queries the `Route` table — each row's last `RouteNode` (by position) supplies the coord.
- Returns entries in no guaranteed order.

GET /api/health

Health check endpoint.

Auth: None

Success (200):

```
{ "status": "ok", "timestamp": "2026-04-20T01:00:00.000Z" }
```

Used by Docker health checks and uptime monitoring.

Portal Endpoints

Security note: These endpoints have **no authentication**. They return session and player demographic data. Access should be restricted to internal networks at the firewall level.

All portal endpoints follow the same pattern:

- Method: POST
 - Input: { sessionId } or { startDate, endDate? }
 - Output: array of session+player data rows
-

POST /api/get-children-adults

Returns party size data for all players in a session.

Auth: None

Request body:

```
{ "sessionId": 202604150 }
```

Success (200):

```
[  
  { "playerId": 1234567890, "numChildren": 2, "numAdults": 1 },  
  { "playerId": 9876543210, "numChildren": 0, "numAdults": 3 }  
]
```

Each row is one `PlayerSession` record with { playerId, numChildren, numAdults }. Rows with `null` values (survey not submitted) are included.

POST /api/get-sessions-playerids-by-child-size

Returns sessions/players filtered by exact child count.

Auth: None

Request body:

```
{ "numChildren": 2 }
```

Success (200):

```
[
  { "sessionId": 202604150, "playerId": 1234567890 }
]
```

POST /api/get-sessions-playersids-by-adult-size

Returns sessions/players filtered by exact adult count.

Auth: None

Request body:

```
{ "numAdults": 1 }
```

Success (200):

```
[
  { "sessionId": 202604150, "playerId": 1234567890 }
]
```

POST /api/get-sessions-playersids-by-family-size

Returns sessions/players filtered by total family size (adults + children).

Auth: None

Request body:

```
{ "familySize": 3 }
```

Success (200):

```
[
  { "sessionId": 202604150, "playerId": 1234567890 }
]
```

Notes:

- Uses a raw SQL `COALESCE` query to handle null values in sum.
 - `COALESCE(num_adults, 0) + COALESCE(num_children, 0) = familySize`
-

POST /api/get-child-adult-data-by-start-end-date

Returns party size data for sessions within a date range.

Auth: None

Request body:

```
{ "startDate": "2026-01-01", "endDate": "2026-03-31" }
```

Field	Type	Required	Notes
startDate	string (ISO date)	Yes	Inclusive lower bound on <code>PlayerSession.createdAt</code>
endDate	string (ISO date)	Yes	Inclusive upper bound on <code>PlayerSession.createdAt</code>

Success (200):

```
[  
  { "sessionId": 202601150, "playerId": 111111111, "numChildren": 1, "numAdults": 2  
}
```

POST /api/get-child-adult-data-to-forever

Returns party size data from a start date through all future records.

Auth: None

Request body:

```
{ "startDate": "2026-01-01" }
```

Success (200): Same shape as date range endpoint.

Notes: No upper bound — returns everything from `startDate` onward.

POST /api/get-child-adult-data-to-end-date

Returns party size data from the beginning of records up to an end date.

Auth: None

Request body:

```
{ "endDate": "2026-03-31" }
```

Success (200): Same shape as date range endpoint.

Notes: No lower bound — returns everything up through `endDate` .

Route Statistics Endpoints

Auth: None (matches existing portal pattern). These are designed for internal portal use.

All 20 Route Statistics endpoints are implemented. Each returns the same envelope shape:


```
{
  "success": true,
  "scope": "session" | "dateRange",
  "scopeValue": <sessionId or { startDate, endDate }>,
  "metric": "<metric-name>",
  "value": <number or object>
}
```

Session endpoints accept: { "sessionId": 202401010 }

Date-range endpoints accept: { "startDate": "2026-01-01", "endDate": "2026-03-30" }
(endDate optional)

Error responses (all endpoints):

Status	Body	When
400	{ "error": "sessionId or startDate is required" }	No valid input
400	{ "error": "Invalid sessionId" }	Non-numeric sessionId
400	{ "error": "Invalid date format – use YYYY-MM-DD" }	Bad date string
500	{ "error": "Failed to compute statistics", "message": "..." }	DB error

Session endpoint	Date-range endpoint	value type
POST /api/route-stats/total-routes-completed-by-session	POST /api/route-stats/total-routes-completed-by-date-range	number
POST /api/route-stats/total-connected-users-by-session	POST /api/route-stats/total-connected-users-by-date-range	number
POST /api/route-stats/users-started-more-than-one-route-by-session	POST /api/route-stats/users-started-more-than-one-route-by-date-range	number
POST /api/route-stats/users-completed-more-than-one-route-by-session	POST /api/route-stats/users-completed-more-than-one-route-by-date-range	number
POST /api/route-stats/users-never-started-route-by-session	POST /api/route-stats/users-never-started-route-by-date-range	number
POST /api/route-stats/users-started-but-not-finished-by-session	POST /api/route-stats/users-started-but-not-finished-by-date-range	number
POST /api/route-stats/total-replays-by-session	POST /api/route-stats/total-replays-by-date-range	{ totalReplays: number, replayUsersCount: number }
POST /api/route-stats/avg-routes-per-user-by-session	POST /api/route-stats/avg-routes-per-user-by-date-range	number

POST /api/route-stats/avg-routes-per-session-by-session	POST /api/route-stats/avg-routes-per-session-by-date-range	number
POST /api/route-stats/replay-ratio-by-session	POST /api/route-stats/replay-ratio-by-date-range	{ byPlayer: number, byRoute: number }

See [ROUTE_STATISTICS_SPEC.md](#) for full metric definitions, query logic, and example responses.